

60 Days - Data Engineering RoadMap

By Srikanth Piniseti (<https://www.linkedin.com/in/srikanthpiniseti/>)

Week 0: Basics of Programming

- Python
 - SQL
-

Week 1: Linux System Administration

Day 1: Introduction to Linux for Data Engineering

- **Key Points:**
 - Overview of Linux distributions (RHEL, CentOS, Ubuntu).
 - Importance of Linux in server management, scripting, and automation.
 - **Troubleshooting:** Common installation issues and solutions (e.g., dependency errors).

Day 2: Package Management

- **Key Commands:**
 - **YUM:** `yum install <package>`, `yum update`, `yum remove`.
 - **DNF:** `dnf install <package>`, `dnf upgrade`, `dnf autoremove`.
 - **RPM:** `rpm -ivh <package.rpm>`, `rpm -e <package>`.
- **Key Points:**
 - Understanding repositories (`/etc/yum.repos.d/`).
 - Importance of keeping packages up to date.
 - **Troubleshooting:** Resolving repository errors and conflicts during installations.

Day 3: User Management

- **Key Commands:**
 - `useradd`, `usermod`, `userdel`.
 - `passwd <username>` for password management.
 - `groupadd`, `gpasswd`.
- **Key Points:**
 - Importance of user roles in data security.
 - Setting up sudoers file with `visudo`.
 - **Troubleshooting:** Issues with user permissions and sudo access (e.g., `visudo` errors).

Day 4: System Logging

- **Key Tools:**
 - `syslog`, `journalctl`, `logrotate`.
- **Key Points:**
 - Importance of logging for auditing and debugging.
 - Configuring `/etc/rsyslog.conf` for centralized logging.
 - **Troubleshooting:** Identifying and resolving common logging issues (e.g., logs not appearing).

Day 5: Network Configuration

- **Key Commands:**
 - `ifconfig`, `ip addr show`, `ip route`.
 - `ping`, `traceroute`, `curl`, `wget`.
- **Key Points:**
 - Configuring network interfaces using `nmcli` or editing `/etc/sysconfig/network-scripts/`.
 - **Troubleshooting:** Diagnosing common network issues (e.g., connectivity problems, IP conflicts).

Day 6: File System Management

- **Key Tools:**
 - `df`, `du`, `mount`, `umount`.
- **Key Points:**
 - Understanding file systems (ext4, xfs, btrfs) and their characteristics.
 - Importance of using LVM (Logical Volume Manager) for storage management.
 - **Troubleshooting:** Resolving mount issues and file system errors.

Day 7: Shell Scripting Basics

- **Key Concepts:**
 - Writing basic scripts: `#!/bin/bash`, variables, loops, and conditionals.
 - **Key Points:**
 - Executing scripts: `chmod +x script.sh`, `./script.sh`.
 - **Troubleshooting:** Common scripting errors and debugging techniques (`set -x`, `set -e`).
-

Week 2: Networking Fundamentals

Day 8: Networking Basics

- **Key Points:**

- Overview of TCP/IP model (Application, Transport, Network, Data Link, Physical).
- Key networking terminology: IP address, subnet mask, gateway.
- **Troubleshooting:** Understanding the impact of misconfigured IP settings.

Day 9: DNS and Domain Management

- **Key Tools:**
 - `dig`, `nslookup`, `host`.
- **Key Points:**
 - How DNS resolution works (recursive vs. authoritative).
 - Setting up and managing a local DNS server with BIND.
 - **Troubleshooting:** Resolving DNS issues (e.g., domain not resolving).

Day 10: Firewalls and Security Groups

- **Key Tools:**
 - `iptables`, `firewalld`, `nftables`.
- **Key Points:**
 - Configuring firewall rules (ALLOW, DENY).
 - **Troubleshooting:** Common firewall problems (e.g., blocked traffic).

Day 11: VPN and Remote Access

- **Key Tools:**
 - OpenVPN, IPsec.
- **Key Points:**
 - Importance of VPN in secure data access.
 - **Troubleshooting:** Common VPN issues (e.g., connection failures).

Day 12: Network Troubleshooting Tools

- **Key Tools:**
 - `ping`, `traceroute`, `netstat`, `tcpdump`.
- **Key Points:**
 - Techniques for diagnosing network issues.
 - **Troubleshooting:** Using `tcpdump` to analyze packet flows and identify problems.

Day 13: Advanced Networking

- **Key Concepts:**
 - Understanding subnets, CIDR notation (e.g., /24).
 - Configuring VLANs for network segmentation.
 - **Troubleshooting:** Addressing common routing issues.

Day 14: Review and Summary

- **Key Points:**

- Recap of key networking concepts and tools covered.
 - Discussing real-world scenarios of network configuration and troubleshooting.
 - Sharing additional resources for learning networking.
-

Week 3: Security Fundamentals

Day 15: Security Overview

- **Key Points:**
 - Importance of security in data engineering.
 - Common security threats (SQL injection, XSS, DDoS).
 - **Troubleshooting:** Identifying security vulnerabilities through audits.

Day 16: Authentication and Authorization

- **Key Methods:**
 - Basic Authentication, OAuth, JWT.
- **Key Points:**
 - Understanding authentication methods (LDAP, Kerberos).
 - **Troubleshooting:** Resolving authentication failures.

Day 17: Active Directory Integration

- **Key Points:**
 - Configuring Linux to authenticate against Active Directory using `sssd`.
 - **Troubleshooting:** Common issues with AD integration (e.g., user not found).

Day 18: Role-Based Access Control (RBAC)

- **Key Points:**
 - Overview of RBAC principles and implementation.
 - **Troubleshooting:** Issues related to permissions and access rights.

Day 19: Data Encryption

- **Key Tools:**
 - GPG, OpenSSL, LUKS for disk encryption.
- **Key Points:**
 - Importance of data encryption in transit and at rest.
 - **Troubleshooting:** Resolving encryption key issues.

Day 20: Auditing and Monitoring

- **Key Tools:**
 - `auditd`, SELinux, `systemd-journald`.

- **Key Points:**
 - Setting up audit logging to track user actions.
 - **Troubleshooting:** Common auditing issues (e.g., log entries missing).

Day 21: Best Security Practices

- **Key Points:**
 - Summary of security best practices for data engineering environments.
 - Importance of regular updates and patch management.
 - **Troubleshooting:** Reviewing logs for compliance and anomalies.
-

Week 4: DevOps for Data Engineers

Day 22: Introduction to DevOps

- **Objectives:**
 - Understand the core concepts of DevOps and its importance in automating and scaling data pipelines.
 - Explore how DevOps enhances data engineering workflows by promoting automation, collaboration, and continuous delivery.
- **Key Topics:**
 - What is DevOps?
 - The role of DevOps in data engineering:
 - Automating data pipeline deployment.
 - Continuous Integration/Continuous Delivery (CI/CD).
 - Benefits for data engineers:
 - Increased agility.
 - Faster iteration of data products.
 - Better collaboration across teams.
- **Tasks:**
 - Study a high-level overview of how DevOps principles apply to data engineering workflows.
 - Map out how automation, CI/CD, and collaboration can benefit specific tasks in data engineering.

Day 23: Version Control with Git

- **Objectives:**
 - Master Git for version control and effective collaboration on data projects.
 - Learn how version control supports the continuous improvement of data pipelines.
- **Key Topics:**
 - Common Git commands: `git init`, `git clone`, `git add`, `git commit`, `git push`, `git pull`, `git branch`, `git merge`.

- Branching strategies: Feature branches, Gitflow, pull requests.
- Handling merge conflicts and rebases.
- **Troubleshooting Focus:**
 - Resolving **merge conflicts**.
 - Managing **rebase issues** in a collaborative team.
- **Tasks:**
 - Set up a Git repository for a data engineering project.
 - Practice resolving merge conflicts using a simple Python-based data pipeline.

Day 24: Continuous Integration (CI) and Continuous Delivery (CD)

- **Objectives:**
 - Learn how to set up and configure CI/CD pipelines for automating data pipelines and reducing manual errors.
 - Understand the importance of automated testing, integration, and deployment for data workflows.
- **Key Topics:**
 - Overview of CI/CD:
 - Continuous Integration: Frequent code commits, automated tests.
 - Continuous Delivery: Automated deployments to production.
 - Common CI/CD tools:
 - **Jenkins**: Automation server for building, testing, and deploying code.
 - **GitHub Actions**: CI/CD directly within GitHub.
 - **Travis CI**: Simple CI for open-source projects.
 - Setting up pipelines for data engineering projects:
 - Data ingestion and processing pipelines.
 - Automated testing for ETL scripts.
- **Troubleshooting Focus:**
 - Debugging **pipeline failures**: Build errors, integration test failures, configuration issues.
- **Tasks:**
 - Create a CI pipeline using Jenkins or GitHub Actions for a data pipeline project.
 - Set up automated tests to validate data transformations before deploying to production.

Day 25: Introduction to Containerization

- **Objectives:**
 - Understand how containers provide a consistent and isolated environment for running data pipelines.
 - Learn the fundamentals of Docker and how to containerize data applications.
- **Key Topics:**
 - Containers vs. Virtual Machines: Lightweight, isolated environments.
 - **Docker** overview:
 - Key commands: `docker build`, `docker run`, `docker ps`, `docker exec`, `docker-compose`.

- Writing Dockerfiles to containerize applications.
- Benefits for data engineers:
 - Consistent environments across development, testing, and production.
 - Easily reproducible and scalable data pipelines.
- **Containers commonly used in data engineering:**
 - **Docker:** Most widely used container platform.
 - **Podman:** Alternative to Docker for containerized applications.
 - **Singularity:** Containers for high-performance computing (HPC).
- **Troubleshooting Focus:**
 - Solving common **Docker issues:** Networking conflicts, permission issues, and resource limits.
- **Tasks:**
 - Write a Dockerfile to containerize a simple Python-based ETL process.
 - Use Docker Compose to manage multiple containers, such as a web scraper and a database.

Day 26: Orchestration Tools for Containers (Kubernetes)

- **Objectives:**
 - Understand how orchestration tools like Kubernetes manage and scale containerized data pipelines.
 - Learn how to deploy and manage containerized data applications using Kubernetes.
- **Key Topics:**
 - **What is orchestration?**
 - Managing multiple containers at scale.
 - Automating the deployment, scaling, and management of containerized applications.
 - **Kubernetes basics:**
 - Core components: Pods, Nodes, Deployments, Services.
 - Kubernetes architecture and Control Plane.
 - Why data engineers need Kubernetes:
 - Easily scale data pipelines based on load.
 - Automate failure recovery and optimize resource use.
- **Relation to Containers:**
 - Kubernetes works with container platforms like Docker to manage the lifecycle of containers in production environments.
 - Orchestration helps automate the scaling and deployment of containerized data applications, ensuring uptime and performance.
- **Common orchestration tools:**
 - **Kubernetes:** The most popular container orchestration platform.
 - **Docker Swarm:** Lightweight alternative to Kubernetes for smaller deployments.
 - **Apache Mesos:** Large-scale cluster management.
- **Troubleshooting Focus:**

- Solving **Kubernetes issues**: Deployment failures, pod crashes, resource constraints.
- **Tasks:**
 - Set up a local Kubernetes environment (using Minikube).
 - Deploy a containerized data pipeline to Kubernetes and scale it based on load.

Day 27: Configuration Management (Puppet & Chef)

- **Objectives:**
 - Learn how configuration management tools automate the configuration of data pipeline environments.
 - Understand the distinction between configuration management and orchestration.
- **Key Topics:**
 - **What is configuration management?**
 - Automating server setup, configurations, and application deployments.
 - **Puppet and Chef overview:**
 - Puppet: Uses manifests to define infrastructure configurations.
 - Chef: Uses cookbooks and recipes for Infrastructure as Code (IaC).
 - Benefits for data engineers:
 - Automate environment setup for consistent and reliable data pipeline deployments.
 - Enforce configuration compliance across different environments (dev, test, production).
- **How it relates to containers:**
 - Configuration management tools like Puppet and Chef can be used to manage the underlying infrastructure where containers are deployed.
 - They ensure that the container runtime (e.g., Docker) and other dependencies are configured properly.
- **Troubleshooting Focus:**
 - Debugging **Puppet/Chef issues**: Resource conflicts, failed runs, syntax errors in manifests or recipes.
- **Tasks:**
 - Create a Puppet or Chef script to automate the setup of a data pipeline environment (e.g., installing dependencies and configuring database connections).

Day 28: Monitoring and Logging for Data Pipelines

- **Objectives:**
 - Set up monitoring and logging tools to track data pipeline performance and detect issues early.
 - Learn how to monitor containerized and orchestrated data applications.
- **Key Topics:**
 - **Monitoring Tools:**
 - **Prometheus**: For collecting and querying metrics from containerized applications.
 - **Grafana**: Visualize metrics from Prometheus and create dashboards.
 - **Logging Tools:**

- **ELK Stack (Elasticsearch, Logstash, Kibana):** Centralized logging for data pipelines.
 - **Fluentd:** Logging aggregator for cloud-native applications.
 - Importance of monitoring for data engineers:
 - Track performance of ETL jobs and data pipelines.
 - Detect failures or performance bottlenecks in real-time.
 - **Relation to Containers and Orchestration:**
 - Prometheus integrates with Kubernetes to monitor containerized applications.
 - Logging solutions can be used to capture logs from both individual containers and orchestrated applications.
 - **Troubleshooting Focus:**
 - Using logs to detect **performance bottlenecks**, **pipeline crashes**, or **resource exhaustion**.
 - **Tasks:**
 - Set up Prometheus and Grafana to monitor a containerized data pipeline running in Kubernetes.
 - Use ELK Stack to analyze logs from multiple containers and troubleshoot errors.
-

Week 5: Big Data Tools Overview

Day 29: Big Data Introduction and Hadoop Ecosystem

- **Objective:** Understand the fundamentals of Big Data and the Hadoop ecosystem.
- **Topics:**
 - **What is Big Data?**
 - Definition and importance
 - Characteristics: Volume, Velocity, Variety, Veracity, Value
 - **Hadoop Ecosystem Overview:**
 - **Hadoop Core Components:**
 - **HDFS (Hadoop Distributed File System):** Storage layer that provides high-throughput access to application data.
 - **YARN (Yet Another Resource Negotiator):** Resource management layer that allocates system resources to various applications running in a Hadoop cluster.
 - **MapReduce:** A programming model for processing large datasets in parallel.
 - **Hadoop Ecosystem Tools:**
 - **Apache Hive:** Data warehouse software that provides data summarization, query, and analysis.
 - **Apache Pig:** High-level platform for creating programs that run on Hadoop.
 - **Apache HBase:** NoSQL database that runs on top of HDFS for real-time read/write access.
- **Activities:**

- Group discussion on the impact of Big Data across various industries (e.g., healthcare, finance, retail).
- Hands-on exercise: Setting up a single-node Hadoop cluster using Cloudera QuickStart or Hortonworks Sandbox.

Day 30: Cluster Management and Monitoring

- **Objective:** Learn about cluster management and monitoring tools in the Hadoop ecosystem.
- **Topics:**
 - **Importance of Cluster Management:**
 - Ensuring reliability, performance, and scalability of the Hadoop cluster.
 - **Cluster Management Tools:**
 - **Apache Zookeeper:** Coordination service for distributed applications; provides distributed synchronization and configuration management.
 - **Apache Ambari:** Web-based tool for provisioning, managing, and monitoring Hadoop clusters.
 - **Cloudera Manager:** A management tool for Cloudera's distribution of Hadoop.
 - **Monitoring Tools:**
 - **Ganglia:** Scalable distributed monitoring system for high-performance computing systems.
 - **Nagios:** Open-source monitoring tool for computer systems and networks.
 - **Grafana:** Analytics and monitoring solution that allows you to visualize metrics from various data sources.
- **Activities:**
 - Demonstration of setting up Zookeeper and its integration with Hadoop.
 - Practical exercise: Using Apache Ambari to monitor cluster performance and resource utilization.

Day 31: Data Ingestion and Streaming Tools

- **Objective:** Explore data ingestion and streaming tools for moving data into Hadoop.
- **Topics:**
 - **Importance of Data Ingestion:**
 - The process of importing data from various sources into Hadoop for analysis.
 - **Data Ingestion Tools:**
 - **Apache Kafka:** A distributed streaming platform used for building real-time data pipelines and streaming applications.
 - **Apache Sqoop:** Tool designed for efficiently transferring bulk data between Hadoop and structured datastores such as relational databases.
 - **Apache Flume:** A distributed, reliable service for efficiently collecting, aggregating, and moving large amounts of log data.
 - **Streaming Data Concepts:**
 - Difference between batch processing and stream processing.

- Use cases for real-time data ingestion (e.g., clickstream analysis, social media data).
- **Activities:**
 - Hands-on: Set up an Apache Kafka cluster and produce/consume messages.
 - Practical exercise: Use Sqoop to import data from a MySQL database into HDFS.

Day 32: Data Storage and Types of Storage Tools

- **Objective:** Understand data storage options in Big Data and their respective tools.
- **Topics:**
 - **Overview of Data Storage Types:**
 - **Structured Data:** Data organized in a predefined manner (e.g., tables).
 - **Semi-structured Data:** Data that does not conform to a fixed schema but still contains tags or markers (e.g., JSON, XML).
 - **Unstructured Data:** Data that lacks a specific format (e.g., text documents, images).
 - **Tools for Data Storage:**
 - **HDFS:** Primary storage system of Hadoop for storing large files across multiple machines.
 - **HBase:** Provides real-time access to large datasets and supports structured data.
 - **Amazon S3:** Cloud storage service for storing and retrieving any amount of data at any time.
 - **Cassandra and MongoDB:** NoSQL databases designed for high availability and scalability.
- **Activities:**
 - Discuss the advantages and disadvantages of different data storage options.
 - Hands-on: Store and retrieve data using HDFS and perform CRUD operations with HBase.

Day 33: Data Processing, Querying, and Analyzing Types and Their Tools

- **Objective:** Learn about data processing, querying, and analysis tools in Big Data.
- **Topics:**
 - **Data Processing Techniques:**
 - Batch processing vs. real-time processing.
 - **Tools for Data Processing:**
 - **Apache Spark:** Fast and general-purpose cluster computing system with in-memory processing capabilities.
 - **Hadoop MapReduce:** Programming model for processing large data sets with a distributed algorithm on a cluster.
 - **Data Querying and Analysis Tools:**
 - **Apache Hive:** SQL-like interface for querying data stored in Hadoop.
 - **Apache Pig:** Scripting language for data processing that compiles to MapReduce.

- **Presto:** Distributed SQL query engine for running interactive analytic queries on data sources of all sizes.
- **Activities:**
 - Hands-on: Write and execute Hive queries to analyze datasets.
 - Practical exercise: Use Spark to process a large dataset and perform transformations.

Day 34: Data Integration, Automation Types, and Their Tools

- **Objective:** Explore data integration tools and automation techniques in Big Data workflows.
- **Topics:**
 - **Importance of Data Integration:**
 - Combining data from different sources into a unified view.
 - **Tools for Data Integration:**
 - **Apache NiFi:** Tool for automating the flow of data between systems; provides a web-based interface to manage data flows.
 - **Apache Airflow:** Workflow management platform for authoring, scheduling, and monitoring data pipelines.
 - **Types of Automation:**
 - ETL (Extract, Transform, Load) processes.
 - Data workflow orchestration and scheduling.
- **Activities:**
 - Demonstration of creating a data flow using Apache NiFi.
 - Hands-on: Set up a simple data pipeline using Apache Airflow to schedule a job.

Day 35: Data Reporting and Visualization

- **Objective:** Understand data reporting and visualization tools for insights.
 - **Topics:**
 - **Importance of Data Reporting and Visualization:**
 - Translating complex data into understandable insights for decision-making.
 - **Tools for Data Reporting and Visualization:**
 - **Tableau:** Interactive data visualization tool that enables users to create a wide range of visualizations.
 - **Power BI:** Business analytics tool by Microsoft that provides interactive visualizations and business intelligence capabilities.
 - **Apache Superset:** Modern data exploration and visualization platform designed to be visual, intuitive, and easy to use.
 - **Techniques for Effective Data Storytelling:**
 - Best practices for creating impactful dashboards and reports.
 - **Activities:**
 - Hands-on: Create a dashboard using Tableau or Power BI to visualize a dataset.
 - Group activity: Present findings using data visualizations and discuss the insights derived from the data.
-

Week 6: ETL Pipelines and Cloud Big Data

Day 36: Introduction to ETL Processes

- **Key Points:**
 - Overview of ETL vs. ELT.
 - Importance of ETL in data integration and transformation.
 - **Troubleshooting:** Identifying bottlenecks in ETL workflows.

Day 37: Building ETL Pipelines

- **Key Tools:**
 - Apache NiFi, Talend, Informatica.
- **Key Points:**
 - Key components of an ETL pipeline (extractors, transformers, loaders).
 - Data quality and validation techniques.
 - **Troubleshooting:** Addressing data quality issues in ETL processes.

Day 38: Data Transformation Techniques

- **Key Points:**
 - Common transformation operations (filtering, aggregation, joining).
 - Importance of data cleansing and enrichment.
 - **Troubleshooting:** Diagnosing transformation errors in pipelines.

Day 39: Data Warehousing Concepts

- **Key Points:**
 - Differences between OLAP and OLTP systems.
 - Importance of star schema vs. snowflake schema.
 - Data warehousing solutions (Snowflake, Amazon Redshift).
 - **Troubleshooting:** Performance tuning of warehouse queries.

Day 40: Cloud Data Engineering

- **Key Points:**
 - Advantages of cloud data engineering (scalability, cost-effectiveness).
 - Overview of cloud providers (AWS, Azure, GCP) and their offerings.
 - **Troubleshooting:** Common issues in cloud resource provisioning.

Day 41: Snowflake Overview

- **Key Points:**
 - Snowflake architecture (storage, compute, services).
 - Importance of separation of compute and storage.
 - Key features (auto-scaling, time travel, zero-copy cloning).
 - **Troubleshooting:** Handling Snowflake query performance issues.

Day 42: Databricks Overview

- **Key Points:**
 - Databricks platform features and capabilities.
 - Importance of collaborative notebooks for data teams.
 - Integration with Delta Lake for data reliability.
 - **Troubleshooting:** Diagnosing issues with cluster performance and job failures.
-

Week 7: Advanced Topics and Best Practices

Day 43: Data Governance

- **Key Points:**
 - Importance of data governance in data engineering.
 - Key principles (data quality, data lineage, metadata management).
 - **Troubleshooting:** Addressing data lineage issues.

Day 44: Data Quality Management

- **Key Points:**
 - Overview of data quality dimensions (accuracy, completeness, consistency).
 - Techniques for monitoring and improving data quality.
 - **Troubleshooting:** Identifying and fixing data quality issues in production datasets.

Day 45: Performance Tuning in Data Engineering

- **Key Points:**
 - Techniques for optimizing ETL performance (parallel processing, indexing).
 - Understanding resource allocation in Hadoop and Spark.
 - **Troubleshooting:** Using monitoring tools to identify performance bottlenecks.

Day 46: Backup and Recovery Strategies

- **Key Points:**
 - Importance of data backup and disaster recovery planning.
 - Techniques for backing up data in Hadoop and cloud environments.
 - **Troubleshooting:** Common recovery process issues (e.g., data loss scenarios).

Day 47: Case Studies in Data Engineering

- **Key Points:**
 - Real-world examples of successful data engineering projects.
 - Lessons learned and best practices from industry leaders.
 - **Troubleshooting:** Discussing challenges faced and how they were overcome.

Day 48: Continuous Learning and Community Involvement

- **Key Points:**
 - Importance of staying updated in the fast-paced data engineering field.
 - Engaging with data engineering communities (meetups, conferences).
 - **Troubleshooting:** Strategies for overcoming learning plateaus.

Day 49: Future Trends in Data Engineering

- **Key Points:**
 - Overview of emerging trends (data mesh, data fabric).
 - The impact of AI and machine learning on data engineering.
 - Preparing for the future: skills to focus on.
 - **Troubleshooting:** Identifying and addressing skills gaps in the team.

Day 50: Review and Summary

- **Key Points:**
 - Recap of the key topics covered over the past 8 weeks.
 - Importance of each topic in the overall data engineering lifecycle.
 - **Troubleshooting:** Reviewing areas where teams commonly struggle.
-